🎓 الاختبار الشامل في UML وOOP – من الملفات الثلاثة

📝 القسم الأول: أسئلة اختيار من متعدد

1. الهدف الرئيسي من استخدام UML هو:
أ) تنفيذ النظام مباشرة
ب) تحسين أداء النظام
ج) توثيق وتحليل وتصميم النظام البرمجي
د) ترجمة الكود تلقائيًا

---

2. أي من المخططات التالية يُستخدم لعرض كيفية تفاعل المستخدم مع النظام؟
أ) Class Diagram
ب) Use Case Diagram

ج) Deployment Diagram
د) State Diagram

---

3. في UML، السهم ذو الرأس المفتوح من صنف إلى صنف آخر يشير إلى:
أ) تجميع (Aggregation)
ب) تركيب (Composition)
ج) وراثة (Inheritance)
د) ارتباط (Association)

---

4. أي من الآتي يُعتبر من مخططات السلوك (Behavioral Diagrams)؟
أ) Class Diagram
ب) Use Case Diagram
ج) Package Diagram

د) Deployment Diagram

---

5. أي مما يلي يحقق التغليف (Encapsulation) في OOP؟
أ) إظهار التفاصيل الداخلية للكائن
ب) إخفاء البيانات داخل الكائن واستخدام واجهات للتحكم بها
ج) منع استخدام الوراثة
د) استخدام الحلقات الشرطية داخل الأصناف

✅ القسم الثاني: صح أو خطأ مع التصحيح إن وجد

6. مخطط النشر (Deployment Diagram) يُستخدم لعرض تسلسل التفاعل بين الكائنات.
7. الكائن هو نسخة حية من الصنف.

8. الوراثة تُساعد على إعادة استخدام الكود.
9. Overloading يسمح بإعادة تعريف دالة من صنف الأب في الابن.
10. Activity Diagram يوضح تدفق الإجراءات داخل عملية معينة.

✏️ القسم الثالث: أسئلة مقالية ورسمية

11. ما الفرق بين الـ Class والـ Object في البرمجة الكائنية؟ مع مثال توضيحي.

12. اشرح باختصار كل من المخططات التالية:

Class Diagram

Use Case Diagram

Sequence Diagram

Activity Diagram

13. ارسم مخطط صنف (Class Diagram) لصنف حساب يحتوي على:

خصائص: اسم العميل، الرصيد

دوال: إيداع، سحب
ثم أنشئ صنفًا حساب_توفير يرث من حساب ويحتوي على دالة إضافة_فائدة.

14. اشرح الفرق بين Overloading وOverriding مع مثال توضيحي لكل منهما.

✅ الإجابات النموذجية

🔷 القسم الأول: اختيار من متعدد

1. ✅ ج – توثيق وتحليل وتصميم النظام

2. ✅ ب – Use Case Diagram

3. ✅ ج – وراثة

4. ✅ ب – Use Case Diagram

5. ✅ ب – إخفاء البيانات واستخدام واجهات تحكم بها

🔷 القسم الثاني: صح أو خطأ

6. ❌ خطأ – مخطط النشر يُستخدم لعرض البنية الفيزيائية للنظام (الأجهزة والبرمجيات المرتبطة بها)

7. ✅ صح – الكائن هو نسخة حية من الصنف

8. ✅ صح – الوراثة تُساعد في إعادة استخدام الكود

9. ❌ خطأ – هذا تعريف Overriding، أما Overloading فهو وجود أكثر من دالة بنفس الاسم ولكن بمعاملات مختلفة

10. ✅ صح – Activity Diagram يُظهر تدفق العمليات خطوة بخطوة

🔷 القسم الثالث: إجابات مقالية

11.

Class: قالب أو مخطط يُحدد الخصائص والدوال.

Object: نسخة من هذا القالب تُستخدم فعليًا في التنفيذ.

> مثال:

} كتاب class
عنوان;
قراءة();
{
كتاب1 = new كتاب(); // كائن

.12

Class Diagram: يُظهر الكائنات (classes)، خصائصها، ودوالها، والعلاقات (مثل الوراثة والتجميع).

Use Case Diagram: يُظهر كيف يتفاعل المستخدم (Actor) مع وظائف النظام (Use Cases).

Sequence Diagram: يُوضح ترتيب الرسائل المتبادلة بين الكائنات بمرور الزمن.

يُوضح تدفق العمليات أو Activity Diagram: الإجراءات بشكل متسلسل، يشبه مخطط الانسياب.

---

13. رسم مبسط:

```
+---------------------+
|        حساب         |
+---------------------+
| - اسم_العميل        |
| - الرصيد            |
+---------------------+
| + إيداع()           |
| + سحب()             |
+---------------------+
          ↑
          |
```

```
+---------------------+
|  حساب_توفير         |
+---------------------+
| + إضافة_فائدة()     |
+---------------------+
```

.14

نفس الدالة تُكتب بأسماء متشابهة :Overloading
.داخل نفس الصنف لكن بمعاملات مختلفة

> مثال:

```
عرض();
عرض(string اسم);
```

عرض(int رقم);

.إعادة تعريف دالة من الأب داخل الابن :Overriding

> مثال:

} طائر class
طيران() { اطبع("يطير بشكل عام"); }
}
} صقر : طائر class
طيران() { اطبع("يطير بسرعة"); }
}